

Upravljanje razvojem informacionih sistema					
Inženjerstvo informacionih sistema					
Dnevnik upotrebe AI tehnologije na projektnom zadatku					
Redni br.	Prompt	Alat	Model	Jezik	Opis dobijenog rezultata
* Primer obrisati kada započnete popunjavanje					
2	Implement the Goal activation state machine: Implement the Goal lifecycle state machine in GoalService end-to-end: Add POST api/Goal/{id}/activate — checks: ≥1 active Strategy (add IsActive bool to Strategy), Assessment with State == Completed exists (via IAssessmentClient), ≥1 GqmGoal exists (via IQgmGoalClient). Transition Status → Active only if all pass, else return 422 with specific error message. Add POST api/Goal/{id}/complete — only valid from Active, transitions to Completed. Add POST api/Goal/{id}/cancel — valid from Draft or Active, transitions to Cancelled. Throw InvalidGoalStateException for illegal transitions. Add unit tests for each transition covering happy path and each failure case.	AntiGravity	Opus 4.6	Engleski	Implementirana state mašina za Goal entitet sa validacijama eksternih servisa i kompletnom pokrivenošću testovima.
3	Refactoring the gqm-goal-service Pagination: I need to refactor the gqm-goal-service to use the standardized pagination logic from our shared module. Specifically, update all relevant interfaces, services, and controllers to replace the local PagerResult<T> implementation with the PaginationRequest and PaginationResponse contracts from Shared.Contracts. After making the changes, write a new unit test for the updated pagination logic and verify that all existing tests for the service still pass.	AntiGravity	Gemini 3 Flash	Engleski	Refaktorisani sistem paginacije u gqm-goal-servisu korišćenjem deljenih ugovora (Shared.Contracts) uz verifikaciju testova.
4	JWT Permission Synchronization: We need to fix the JWT permissions and ensure all authorization checks align with the seeded permissions in the user-service. Please review the [RequirePermission] attributes across the controllers and correct any mismatched permission names. Then, update Permissions.cs and DataSeeder.cs in the user-service to include any missing permissions and assign them to the correct roles. Make sure the controllers are using the exact string constants defined in the seeder.	AntiGravity	Gemini 3 Flash	Engleski	Sinhronizovane permisije između baze, seeder-a i kontrolera, čime je osigurana konzistentna RBAC autorizacija.
5	You have full project context (AntiGravity enabled). Carefully analyze the entire project... You are implementing the Department Service microservice. [Verbatim prenešen zahtev za analizu domena, granica mikroservisa, implementaciju CRUD-a, paginacije, .NET 10 stack-a, AutoMapper-a i Docker konfiguracije].	AntiGravity	Gemini 3 Flash	Engleski	Izvršena detaljna analiza i implementiran kompletan Department mikroservis sa svim slojevima i Docker podrškom.
6	I suggest defining an IApplicationDbContext interface within the Application layer instead of injecting the concrete DbContext here. This aligns with the Dependency Inversion Principle, ensuring the Application layer defines its own data requirements while keeping the Infrastructure details decoupled. It also makes our unit tests much cleaner since we can easily mock the interface.	AntiGravity	Gemini 3 Flash	Engleski	Primena DiP principa uvođenjem interfejsa za bazu podataka u Application sloj radi lakšeg testiranja i dekaplovanja.
7	Scan department-service and check if everything is okay and working. If you find smth violating clean code and optimizations, tell me	AntiGravity	Gemini 3 Flash	Engleski	Revizija koda Department servisa uz ispravke Clean Code propusta i optimizaciju performansi.

8	You are working on a microservice-based application with the following stack: Backend: .NET microservices Frontend: Angular The project currently has a partial authentication implementation, but the frontend does not fully integrate with backend APIs to manage the user session. Your task is to implement a complete end-to-end authentication flow between frontend and backend. Do NOT start coding immediately. First: What do you think about this implementation plan? Also, what do you think about this Task plan: User Service Implementation Planning * Study existing service architecture (department-service as reference) * Create implementation plan * Get user approval on plan Domain Layer * Create UserService.Domain project * Create	AntiGravity	Gemini 3 Flash	Engleski	Implementiran kompletan auth tok sa Access/Refresh tokenima, Angular interceptorom i sigurnim čuvanjem sesije.
9	What do you think about this implementation plan? Also, what do you think about this Task plan: User Service Implementation Planning * Study existing service architecture (department-service as reference) * Create implementation plan * Get user approval on plan Domain Layer * Create UserService.Domain project * Create	AntiGravity	Sonnet 4.6	Engleski	Ovaj odgovor predstavlja kritičku reviziju plana koja identifikuje ozbiljne propuste u arhitekturi, pre svega nedostatak konkretnih koraka za OAuth/JWT autentifikaciju i neusklađen broj korisničkih uloga. Njegov cilj je da te spreči da pošalješ nepotpun prompt AI modelu, obezbeđujući da tvoj user-service dobije svu potrebnu logiku (poput refresh tokena i hešovanja lozinki) iz prvog pokušaja.
10	README.md I want to create GQM+Strategies app. Follow structure from README.md file that I attached. My team already made access-service, assessment-service, department-service, goal-service, gqm-goal-service, premise-service and you can see that services in folder services. I need to create user-service where I will have main entities User, Role, Permission but you can see part of the model on picture that I attached. I also attached whole model so you can have more context for roles and permission and logic. I want you to create me layer by layer so I can commit changes. Create me Layers in this order: Domain Layer, Infrastructure Layer, Application Layer, API Layer, Tests Layer. I want to have 4 Roles: System Admin, Organization admin, Departmen manager and Viewer. I sent you picture with permissions for each role. If you think something should be added then tell me. Also if you cannot see some folder tell me Should I change something or add some comment?	AntiGravity	Opus 4.6	Engleski	Ovaj odgovor predstavlja završnu validaciju plana i pripremu za kôdiranje, gde AI (Opus) preuzima ulogu preciznog arhitekta. Fokus je na otklanjanju dvosmislenosti u nazivima i broju uloga kako bi prva iteracija koda (Domain Layer) bila 100% tačna i usklađena sa ostatkom sistema.
11	check me for this if needed. This is suggestion: UserOrganizationRoleResponse includes RoleNameThis is a smart addition. Instead of forcing the client to make a second call to resolve the role name from RoleId, it's included directly. The AutoMapper profile will need to map this from the Role navigation property, so make sure to includes .Include(uor => uor.Role) in any query that returns this DTO. Also, check me all the changes that I made and run build again	AntiGravity	Opus 4.6	Engleski	Ovaj odgovor predstavlja tehničku validaciju i "bug-fix" fazu u kojoj AI deluje kao strogi Code Reviewer. On potvrđuje tvoje arhitektonske optimizacije, ali istovremeno ispravlja kritične sintaksičke greške (poput metoda van tela klase) i nedostajuće using direktive, čime osigurava da je projekat stabilan (Build passed) pre prelaska na API nivo.
12	I found that ResetPassword tests are missing entirely. I added them in AuthAppServiceTests but check me comment that I left on task. The invalid token test is straightforward. The valid token test has a real problem - ForgotPasswordAsync stores the SHA-256 hash of the token but never returns the plain token, so there's no way to test a successful reset without either exposing the plain token or making ForgotPasswordAsync return it. Make ForgotPasswordAsync return the plain token specifically for testability. Also, check me all the tests and changes that I've made.	AntiGravity	Gemini 3 Flash	Engleski	Postignuto je usklađivanje kôda sa novim testovima kroz promenu povratnog tipa ForgotPasswordAsync metode, čime je omogućeno testiranje uspešnog resetovanja lozinke bez narušavanja bezbednosti API-ja. Odgovor potvrđuje ispravnost tvojih naprednih test slučajeva (poput Moq autentifikacije i BCrypt verifikacije) i garantuje stabilnost sistema sa svih 120 položenih testova nakon popravke sitnih sintaksičkih grešaka.
13	I was wondering, should I create enums for permissions and roles in code even they are in the db?	AntiGravity	Gemini 3 Flash	Engleski	Arhitektonska odluka o korišćenju static const string klasa za role i permisije radi sprečavanja "magic strings" grešaka.

14	Can you generate me final table with roles and permissions	AntiGravity	Sonnet 4.6	Engleski	Generisana finalna matrica pristupa koja definiše prava za svih 5 uloga u sistemu (System Admin do Viewer-a).
15	Potrebna mi je tvoja pomoć da se pripremim, analiziram trenutni status projekta i počnem sa radom na svom mikroservisu. Projekat je predviđen da bude radjen u .NET u mikroservisnoj arhitekturi. Rezultat rada treba da bude proizvod u vidu softvera koji pruza podrsku za donosnje, pracenje i analizu ciljeva i njihovog medjusobnog uticaja koristeći GQM+ strategiju. Radjen je domain driven razvoj sto je podrazumevalo istrazivanje domena i razumevanje GQM+ strategije i kako ona moze biti inkorporirana u ovakav sistem. Kreiran je model i definisani su entiteti kao i podela u agregate tj mikroservise. Sistem ce voditi evidenciju o sledecim entitetima: ORGANIZACIONI DEO SISTEMA 1. ORGANIZATION Svrha entiteta Predstavlja organizaciju (npr. kompaniju, instituciju) unutar koje postoje departmani, korisnici i ciljevi. Atributi * id: UUID – jedinstveni identifikator organizacije * name: String – naziv organizacije * description: String – opis organizacije Veze * 1 : N sa Department * Jedna organizacija ima više departmana	AntiGravity	Sonnet 4.6	Srpski	Potvrđeno duboko razumevanje domena i GQM+ strategije, mapirani svi entiteti i veze unutar 6 ključnih agregata, i definisane inicijalne smernice za uspostavljanje razvojnog okruženja prema README pravilima.

16	Moj rad treba da bude usmeren ka rad na dva agregata: PREMISE i GOAL PROBABILITY ASSESMENT. Drugi članovi tima rade na drugim agregatima i kao konacno resenje treba da dobijemo funkcionalnu mikroservisnu aplikaciju. bitno mi je da dobijem konkretne, precizne i tacne smernice u strukturi koje su u skladu sa utvrdjenim pravilima za celi projekat, kako bi se na kraju dobila usaglasena celina. Za sada je najbitnije da fokus bude na readme fajlu i pravilima koja su tamo definisana. Pre nego sto napravis izmene u kodu, napisi mi prvo tvoj plan rada, a tek onda nakon sto ja pregledam i utvrdim da je to ono sto zelim, prelazimo na stvarne izmene u kodu i projektu Kao dio api dokumentacije za moje aggregate izdvajam ove zahteve: Premise POST /premises — Create Premise Request Body: { "description": "Team has cloud expertise", "type": "ASSUMPTION", "goalId": "uuid", "strategyId": "uuid"	AntiGravity	Sonnet 4.6	Srpski	Kreiran detaljan tehnički plan implementacije za mikroservise Premise i Goal Probability Assessment, definisani ugovori za sve API endpointe (uključujući napredno filtriranje) i uspostavljena logika za verziranje premisa (izmena verzije umesto klasičnog update-a).
17	Act as a Senior Software Architect specialized in distributed systems, .NET microservices, SaaS multi-tenant architecture and Domain-Driven Design. Your task is to deeply analyze the entire codebase of this project and produce a structured architectural review and implementation roadmap. The system is a multi-tenant SaaS platform for Strategic Goal Management based on GQM+ methodology. It includes the following domain concepts: Goal, GQMGoal, Question, Target, Measurement, Strategy, Premise (Context/Assumption), GoalProbabilityAssessment, Department, Organization, User, Role, Permission, and Transaction/Audit logging. The project uses .NET and Docker and currently has multiple microservices but no API Gateway. Your objective is to: 1. Analyze the current project structure and architecture. 2. Identify how to implement services communication. 3. Evaluate how to implement authentication and authorization mechanisms. 4. Evaluate how to implement transaction consistency and logging. 5. Identify architectural risks and technical debt. 6. Provide a clear next-step implementation roadmap tailored to THIS project. --- ANALYSIS STRUCTURE REQUIRED IN RESPONSE Your response must be structured in the following sections: Project Structure Overview	AntiGravity	Sonnet 4.6	Engleski	Izrađena sveobuhvatna arhitektonska revizija sistema sa fokusom na SaaS multi-tenancy, predložen model za YARP API Gateway i međuservisnu komunikaciju, identifikovani kritični bezbednosni rizici i postavljen precizan Roadmap za prelazak projekta u produkciju spremnost.

18	Design an Analytics page for an enterprise multi-tenant B2B SaaS web application used for Strategic Goal Management based on the GQM+Strategies methodology. The application allows organizations to define, refine, measure and analyze strategic goals, where goals form a hierarchical structure derived through strategies. The hierarchy behaves as a Directed Acyclic Graph (DAG) but is visually represented as a tree starting from a root goal, and there must never be cycles in the goal structure. Use the existing enterprise design system: 1440px desktop-first layout, Inter typography, 8px spacing grid, 8px border radius, soft shadows, palette navy #1E2A38, indigo #3F51B5, teal #2CB1A1, background #F7F9FC, and white surfaces. The page uses the standard application layout consisting of sidebar navigation, topbar with organization switcher and user menu, and a main workspace area. The Analytics module is accessible to users with the view_analytics permission and is read-only. The page should be designed as a two-column analytics workspace, where the left side (~70% width) contains an interactive goal hierarchy visualization and the right side (~30% width) contains an analytics dashboard with KPIs and charts. At the top of the page include a filter bar for analytics scope selection with dependent dropdowns: Department selector, and Root Goal selector. When a user navigates to this page with current organization the system loads the available departments, when a department is selected the system loads root goals belonging to that department, and when a root goal is selected the application fetches and renders the goal hierarchy tree. If only organization or department is selected, analytics should update based on that scope. The filter bar should include a Reset Filters button and loading skeletons while data is fetched. The central visualization component is the Goal Hierarchy Graph, which renders the structure of goals and strategies as a top-to-bottom hierarchical tree layout. The graph must support unlimited depth, because strategies can generate new goals which may again generate further strategies and goals. The visualization includes two node types: Goal nodes and Strategy nodes, connected with edges that represent Goal → Strategy → Child Goal relationships. A goal can have multiple strategies, a strategy can produce multiple child goals, and a goal can originate from at most one strategy. The graph must visually emphasize hierarchy and avoid edge overlaps. The graph should support zooming, panning, node dragging, fit-to-screen controls, expand/collapse of subtrees, and a mini-map overview. Hovering over nodes shows a tooltip with summary information, while clicking a node opens a right-side drawer panel with detailed information. Goal nodes represent the Goal entity and appear as rounded cards displaying key attributes from the domain model: Focus, Object, Status badge, and BaselineProbability indicator. The status may be Draft, Active, Achieved, or Failed, and each status should have a distinct color indicator (Draft gray, Active indigo, Achieved green, Failed red). The node should also display a small probability indicator representing the BaselineProbability value between 0 and 1. Example goal node content shows the focus and object combined into a readable sentence, such as "Develop the marketability for IP testing products," along with status and probability. Strategy nodes represent the Strategy entity and appear as rounded cards displaying key attributes from the domain model: Focus, Object, Status badge, and BaselineProbability indicator. The status may be Draft, Active, Achieved, or Failed, and each status should have a distinct color indicator (Draft gray, Active indigo, Achieved green, Failed red). The node should also display a small probability indicator representing the BaselineProbability value between 0 and 1. Example strategy node content shows the focus and object combined into a readable sentence, such as "Develop the marketability for IP testing products," along with status and probability.	AntiGravity	Sonnet 4.6 i FigmaAI	Engleski	Svrha ovog prompta je da AI-ju pruži kompletnu specifikaciju za dizajn Analytics stranice u enterprise SaaS aplikaciji za upravljanje strateškim ciljevima zasnovanoj na GQM+Strategies metodologiji. Prompt objašnjava poslovni kontekst sistema, strukturu hijerarhije ciljeva i strategija, kao i način na koji se ti podaci trebaju vizualizovati. Takođe definiše raspored stranice, interakcije korisnika i analitičke komponente poput grafova, KPI kartica i filtera. Cilj je da Figma AI ili Sonnet generiše realističan i funkcionalan UI dizajn koji odgovara stvarnoj logici aplikacije.
----	--	-------------	----------------------	----------	---

Link do celog sheet-a: https://onedrive.live.com/?xc=/personal/20b3b3d254b8f6f9/IQDJ1aofVFKDSyGRpIUHH5EsAawo8fIjX6Owa2F3K4IW0go?rttime=yypWo_-R-3kg&redeem=aHR0cHM6Ly8xZjJLm1Zl3gVYy8yMGZyYjNkMjU0YjhmNmY5L0lRRGoxYW9GvKZLRFNZZ1JwaVVISDVFc0Fhd29CZmlqWDZPd0EyRjNLElXMGdvP2U9Qnkt0s2